物体边缘检测

物体边缘检测
- 例程实验效果简介
- 代码讲解
  - 关键代码
  - 代码结构
- 边缘检测函数
  - find_edge()

例程实验效果简介

> **【边缘检测】** 是图像处理和计算机视觉中的基本问题，【边缘检测】的目的是标识数字图像中亮度变化明显的点。图像属性中的显著变化通常反映了属性的重要事件和变化。

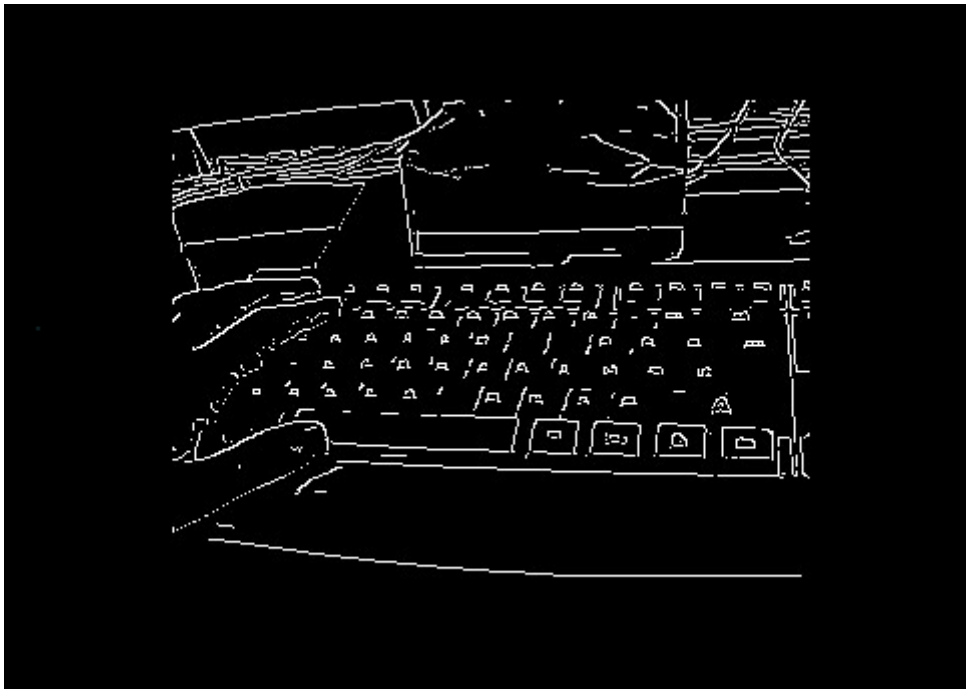
本节例程代码位于：【源码汇总 / 04.Detecting / 04.edges.py】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

点击CanMV IDE左下角的运行按钮，

将K230的摄像头对准周围的物体

可以看到屏幕上会把物体的边缘用白线标记出来



代码讲解

本节我们要用的的外设主要是摄像头模块

线段检测由 K230中的 find_edges() 方法实现，该方法属于image模块

关键代码

```
# 初始化设备 / Initialize devices
sensor = init_sensor()
init_display()
MediaManager.init()
sensor.run()

# 初始化时钟 / Initialize clock
clock = time.clock()

# 计算显示偏移量以居中显示 / Calculate display offsets for center alignment
x_offset = round((DISPLAY_WIDTH - FRAME_WIDTH) / 2)
y_offset = round((DISPLAY_HEIGHT - FRAME_HEIGHT) / 2)

while True:
    clock.tick()  # 更新时钟 / Update clock

    # 捕获和处理图像 / Capture and process image
    img = sensor.snapshot()
    process_image(img)

    # 居中显示图像 / Display image in center
    Display.show_image(img, x=x_offset, y=y_offset)

    # 显示FPS / Display FPS
    print(clock.fps())
```

代码结构

配置部分：

- 定义了两组分辨率：传感器捕获分辨率(1920x1080)和实际处理的帧分辨率(640x480)，如果需要提升帧率，可以将实际处理的分辨率降低。
- 显示屏分辨率设置为640x480

初始化函数：

- `init_sensor()`：配置摄像头传感器，设置分辨率和灰度图像格式
- `init_display()`：初始化显示屏

图像处理：

- `process_image()`：使用Canny算法进行边缘检测，阈值设置为50-80

主程序流程：

- 初始化所有设备（传感器、显示屏）
- 计算显示偏移量，使图像在屏幕上居中显示
- 进入主循环：
  - 捕获图像
  - 进行边缘检测处理
  - 在屏幕上显示处理后的图像
  - 显示当前FPS（每秒帧数）

错误处理：

- 包含了完整的异常处理机制
- 程序结束时会进行资源清理（停止传感器、清理显示、垃圾回收）

边缘检测函数

find_edge()

```
image.find_edges(edge_type[, threshold])
```

该函数将图像转换为黑白图像，仅保留边缘为白色像素。

- `edge_type` 可选值包括：
  - `image.EDGE_SIMPLE` - 简单的阈值高通滤波算法
  - `image.EDGE_CANNY` - Canny 边缘检测算法
- `threshold` 是包含低阈值和高阈值的二元元组。您可以通过调整该值来控制边缘质量，默认设置为 `(100, 200)`。

注意： 此方法仅支持灰度图像。